MZ2POL-03: Assessment and Migration Planning

> **Series:** MZ2POL — Management Zone Migration: Segments, Policies, Workflows | **Notebook:** 4 of 14 | **Created:** December 2025 | **Last Updated:** 10/13/2026

Overview

This notebook guides you through a comprehensive assessment of your current Management Zone implementation and helps you create a detailed migration plan to the new Policies, Boundaries, and Segments model.

Table of Contents

1. Management Zone Inventory
2. MZ Pattern Classification
3. Security Context and Bucket Planning
4. Migration Priority Matrix
5. Migration Timeline Template
6. Dependencies Checklist
7. Deliverables Checklist
8. Additional Resources

Prerequisites

- Completed MZ2POL-01 and MZ2POL-02
- Admin access to Dynatrace environment

- Access to Account Management
- List of current user groups and their MZ assignments

Learning Objectives

By the end of this notebook, you will:
1. Have a complete inventory of your Management Zones
2. Understand the mapping from MZs to new constructs
3. Have a prioritized migration plan
4. Know the risks and mitigation strategies

---

1. Management Zone Inventory

> ⚠️ **Important:** Management Zone configurations **cannot be queried via DQL**.
>
> **Options for MZ inventory:**
> 1. **Dynatrace UI (Recommended):** Settings → Management Zones
> 2. **SDK Analysis:** See **MZ2POL-00: SDK Management Zone Analysis Tool** for comprehensive analysis including rule patterns, coverage metrics, and migration readiness.

Entity Distribution Across MZs

> ⚠️ **Limitation:** DQL queries against `managementZones` may not accurately reflect the full MZ entity distribution due to how MZ rules propagate. For accurate counts, use the **classic Dynatrace UI** which shows entity counts directly on each Management Zone configuration.

The query below provides an approximation of entity distribution:

```
// Service distribution by Management Zone
fetch dt.entity.service
| expand mz = managementZones
| summarize serviceCount = count(), by:{managementZone = mz}
| sort serviceCount desc
| limit 25

// Note: dt.entity.* is deprecated, but this is a migration-ASSESSMENT query — it inspects
// the classic constructs this migration replaces, so keep the classic query above:
//   - management zones have NO Smartscape node equivalent; they are what this migration
//     replaces with segments and policies.
```

---

2. MZ Pattern Classification

Categorize Your Management Zones

Group your MZs by their primary purpose:

Category 1: Team/Ownership Based

- **Pattern**: MZs named after teams (e.g., "Team-Frontend", "Platform-Team")
- **Migration**: → Security Context + Policies + Boundaries
- **Priority**: High (affects user access)

Category 2: Environment Based

- **Pattern**: MZs for environments (e.g., "Production", "Staging", "Development")
- **Migration**: → Boundaries with environment conditions
- **Priority**: High (critical for access separation)

Category 3: Geographic/Region Based

- **Pattern**: MZs for regions (e.g., "NA-East", "EU-West", "APAC")
- **Migration**: → Segments with cloud region/location filters
- **Priority**: Medium (primarily filtering)

Category 4: Application/Service Based

- **Pattern**: MZs for applications (e.g., "Payment-System", "User-Portal")
- **Migration**: → Segments with service/application filters
- **Priority**: Medium (primarily filtering)

Category 5: Infrastructure Based

- **Pattern**: MZs for infrastructure (e.g., "Kubernetes", "Legacy-VMs")
- **Migration**: → Segments with infrastructure type filters
- **Priority**: Low (usually informational)

---

3. Security Context and Bucket Planning

Design Your Security Context Strategy

Security Context enables entity-level access control. Plan how to tag your entities:

Entity Type	Security Context Value	Purpose
Services	`team-{teamname}`	Team ownership
Hosts	`env-{environment}`	Environment separation
Applications	`app-{appname}`	Application isolation

Design Your Bucket Strategy

> ⚠️ **Critical:** Plan buckets carefully - names are immutable and data cannot be moved between buckets.

Bucket Decisions:

Decision	Options	Considerations
Partitioning dimension	Team, Environment, Region, Compliance	What drives access control?
Naming convention	`{team}_{datatype}_{retention}`	Include org, type, and retention
Number of buckets	Per team? Per environment? Both?	Balance isolation vs management overhead

Bucket Limitations to Consider:

Limitation	Impact on Planning
One data type per bucket	Need separate buckets for logs, spans, metrics
Maximum 80 buckets per environment	Plan for scale within this limit
~1 TB/day optimal per bucket	Best query performance
1-3 TB/day acceptable	Limited query window due to 500 GB scan limit
No documented hard cap per bucket	Dynatrace sizing guidance on where to split differs between pages (Partition data (DT docs), Logs bucket strategy (DT docs))
Names immutable	Choose naming convention carefully upfront
No data migration	Cannot consolidate or split buckets later

Query Constraints (affects bucket sizing):

Constraint	Value
Maximum data scanned per query	500 GB
Maximum records returned	1,000
Maximum response payload	1 MB

Mapping MZs to Buckets

For each MZ, determine if bucket partitioning is needed:

MZ Pattern	Bucket Recommendation
Team-based MZ with strict isolation needs	Create team-specific buckets
Environment MZ (Prod/Dev separation)	Consider environment buckets
Compliance MZ (PCI, HIPAA)	**Strongly recommended** - physical separation
Regional MZ (filtering only)	Usually **not needed** - use Segments instead
Application MZ (informational)	Usually **not needed** - use Segments instead

> **For comprehensive bucket guidance**, see **ORGNZ-03: Bucket Strategy and Design**.

Query Entities Without Security Context

Identify entities that need security context assignment:

```
// Find services without security context
// These need to be tagged before migration
fetch dt.entity.service
| filter isNull(dt.security_context)
| fields entity.name,
         managementZones,
         tags
| limit 50

// Note: dt.entity.* is deprecated, but this is a migration-ASSESSMENT query — it inspects
// the classic constructs this migration replaces, so keep the classic query above:
//   - management zones have NO Smartscape node equivalent; they are what this migration
//     replaces with segments and policies.
//   - dt.security_context is an ARRAY on Smartscape nodes (empty [] when unset, not null),
//     so isNull / isNotNull(dt.security_context) does not carry over — a Smartscape rewrite
//     would miscount coverage.
//   - entity tags are not a flat "tags" field on Smartscape (resolve via getNodeField).
```

```
// Count entities by security context presence
// Helps estimate security context tagging effort
fetch dt.entity.service
| summarize
    withContext = countIf(isNotNull(dt.security_context)),
    withoutContext = countIf(isNull(dt.security_context))

// Note: dt.entity.* is deprecated, but this is a migration-ASSESSMENT query —
it inspects
// the classic constructs this migration replaces, so keep the classic query
above:
//   - dt.security_context is an ARRAY on Smartscape nodes (empty [] when
unset, not null),
//     so isNull / isNotNull(dt.security_context) does not carry over — a
Smartscape rewrite
//     would miscount coverage.
```

---

4. Migration Priority Matrix

Prioritize Based on Impact and Complexity



Priority	Criteria	Examples
P1 - Critical	Security/compliance requirements	Production access control

Priority	Criteria	Examples
P2 - High	Actively used for access control	Team-based MZs
P3 - Medium	Used for filtering only	Region/app MZs
P4 - Low	Informational/rarely used	Legacy MZs

Risk Assessment

Risk	Likelihood	Impact	Mitigation
Users lose access	Medium	High	Parallel running period
Wrong data visible	Medium	High	Thorough testing
Alert routing breaks	High	Medium	Update alerts before cutover
Dashboard filters fail	Medium	Low	Pre-update dashboards

---

5. Migration Timeline Template

Recommended Phases

Phase 1: Preparation

- [ ] Complete MZ inventory
- [ ] Document current user access patterns
- [ ] Create migration mapping for all MZs
- [ ] Design security context strategy
- [ ] **Design bucket strategy** (if using buckets for isolation)
- [ ] Identify dependencies (alerts, dashboards, APIs)

Phase 2: Foundation - Buckets and Security Context

- [ ] **Create Grail buckets** (if applicable)
- [ ] **Configure OpenPipeline routing** to buckets
- [ ] Apply security context to entities

- [ ] Create user groups in Account Management
- [ ] Set up IAM policy structure
- [ ] Create boundary definitions

> ⚠️ **Bucket Order:** Create buckets and configure pipeline routing BEFORE migrating policies. Data must flow to correct buckets first.

Phase 3: Segments

- [ ] Create segments for data filtering
- [ ] Test segment functionality
- [ ] Update dashboards to use segments
- [ ] Validate filtering behavior

Phase 4: Policy Assignment

- [ ] Create bucket-based policies (if using buckets)
- [ ] Bind policies to groups
- [ ] Apply boundaries to policy bindings
- [ ] Test user access (parallel with MZs)
- [ ] Document access verification results

Phase 5: Cutover

- [ ] Rebuild MZ-scoped alerting profiles as problem-triggered workflows (affected-entity tags + DQL matcher) — **not** as segments
- [ ] Migrate API integrations
- [ ] Remove MZ-based access (RBAC)
- [ ] Monitor for access issues

Phase 6: Cleanup

- [ ] Verify no MZ dependencies remain
- [ ] Archive MZ configurations
- [ ] Remove unused MZs
- [ ] Update documentation

Bucket Migration Checklist

If using buckets for team/compliance isolation:

- [ ] Bucket naming convention finalized
- [ ] Buckets created for each data type (logs, spans, metrics)
- [ ] OpenPipeline routing rules configured
- [ ] Data flowing to correct buckets verified
- [ ] Bucket-specific policies created
- [ ] Policies include `WHERE storage:bucket-name = "..."` conditions
- [ ] Boundaries include `storage:bucket-name IN (...)` restrictions

---

6. Dependencies Checklist

What References Management Zones?

Before migrating, identify all MZ dependencies:

Alerting

- [ ] Alerting profiles using MZ filters
- [ ] Problem notifications scoped to MZs
- [ ] Metric events with MZ dimensions

Dashboards & Reports

- [ ] Dashboards with MZ filters
- [ ] Reports scoped to MZs
- [ ] SLOs using MZ-filtered data

Integrations

- [ ] API calls with MZ parameters
- [ ] Automation workflows using MZ scopes
- [ ] Third-party integrations filtering by MZ

Access Control

- [ ] User groups with MZ assignments
- [ ] RBAC roles using MZ restrictions

Summary

In this notebook, you:

1. **Inventoried** all Management Zones with DQL queries
2. **Classified** MZs by pattern (team, environment, region, etc.)
3. **Created** migration mapping templates
4. **Planned** security context strategy
5. **Prioritized** migration based on risk and impact
6. **Identified** dependencies to address

Deliverables Checklist

- [ ] Complete MZ inventory spreadsheet
- [ ] Migration mapping document for each MZ
- [ ] Security context assignment plan
- [ ] Prioritized migration schedule
- [ ] Risk assessment and mitigation plan
- [ ] Dependencies checklist

Next Steps

First, the tagging foundation. Every replacement for a management zone reads a field on the data, so the tags have to be in place before you build on them:
- **MZ2POL-10: Tagging at Source** — the model change, the enrichment ladder, and the readiness gate
- **MZ2POL-11: Translating MZ Rules into Source Attributes** — the rule-by-rule catalog that turns this inventory into target fields

- **MZ2POL-12: IAM Reference Architecture** — how the access-control zones become groups, default policies and one boundary per team, and the boundary pitfalls that widen access
- **MZ2POL-14 [WORKSHOP]: Zone Disposition** — run the decisions in this notebook with the zone owners; **MZ2POL-95 [LAB]** is the worked example on a sample estate

Then continue to **MZ2POL-04: Policies and Boundaries** to:
- Learn policy syntax and structure
- Create custom policies
- Understand default policy capabilities

Additional Resources

- Management Zones Documentation
- Upgrade from RBAC to IAM Policies
- Grant Access to Entities with Security Context

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.